

Lab-04: Merge Conflict Resolution Hands-On Lab

Aim

The aim of this lab is to understand how merge conflicts occur in Git and learn how to resolve them when the same file is modified in different branches. This lab demonstrates creating a conflict, identifying it, resolving it manually, committing the resolved changes, and maintaining the repository.

Objectives

- Understand merge conflicts in Git.
- Create conflicting changes in different branches.
- Resolve merge conflicts manually.
- Commit the resolved changes.
- Update .gitignore.
- Delete the merged branch.

Prerequisites

- Git installed and configured.
- Existing Git repository (GitDemo).
- GitHub account.
- Command Prompt or Git Bash.

What is a Merge Conflict?

A merge conflict occurs when the same file is modified differently in two branches, and Git cannot automatically determine which changes should be kept. The conflict must be resolved manually before completing the merge.

Implementation

Step 1: Open the Existing Repository

```
cd Desktop
```

```
cd GitDemo
```

Verify the repository status.

```
git status
```

Output:

On branch main

nothing to commit, working tree clean

Step 2: Create a New Branch

```
git branch GitWork
```

Switch to the new branch.

```
git checkout GitWork
```

Step 3: Create a File

Create a file named **hello.xml**.

```
echo <message>Hello from GitWork</message> > hello.xml
```

Verify.

```
type hello.xml
```

Output:

```
<message>Hello from GitWork</message>
```

Check status.

```
git status
```

Step 4: Commit the Branch Changes

```
git add hello.xml
```

```
git commit -m "Added hello.xml in GitWork"
```

Step 5: Switch to Main Branch

```
git checkout main
```

Step 6: Create the Same File with Different Content

```
echo <message>Hello from Main Branch</message> > hello.xml
```

Verify.

```
type hello.xml
```

Output:

```
<message>Hello from Main Branch</message>
```

Commit the file.

```
git add hello.xml
```

```
git commit -m "Added hello.xml in main"
```

Step 7: View Commit History

```
git log --oneline --graph --decorate --all
```

Git displays both branches and their commits.

Step 8: Compare Both Branches

```
git diff main GitWork
```

Git displays the differences between both versions of hello.xml.

Step 9: Merge the Branch

```
git merge GitWork
```

Git reports a merge conflict.

Example:

Auto-merging hello.xml

CONFLICT (content): Merge conflict in hello.xml

Automatic merge failed; fix conflicts and then commit the result.

Step 10: View the Conflict

Open the file.

type hello.xml

Output:

```
<<<<<<< HEAD
<message>Hello from Main Branch</message>
=====
<message>Hello from GitWork</message>
>>>>>>> GitWork
```

These markers indicate the conflicting changes.

Step 11: Resolve the Conflict

Replace the contents of hello.xml with:

```
<message>Hello from Main Branch</message>
<message>Hello from GitWork</message>
```

Save the file.

Step 12: Add the Resolved File

```
git add hello.xml
```

Step 13: Complete the Merge

```
git commit -m "Resolved merge conflict"
```

Step 14: Verify Repository Status

```
git status
```

Output:

On branch main

nothing to commit, working tree clean

Step 15: Ignore Backup Files

Append backup file extensions to .gitignore.

```
echo *.bak >> .gitignore
```

Verify.

```
type .gitignore
```

Output:

```
*.log
```

```
log/
```

```
*.bak
```

Commit the changes.

```
git add .gitignore
```

```
git commit -m "Updated gitignore for backup files"
```

Step 16: List All Branches

```
git branch
```

Output:

```
* main
```

```
GitWork
```

Step 17: Delete the Branch

```
git branch -d GitWork
```

Output:

Deleted branch GitWork.

Step 18: View Commit History

```
git log --oneline --graph --decorate
```

Git displays the complete commit history, including the merge commit.

Step 19: Push to GitHub

```
git push origin main
```

Commands Executed

```
cd Desktop
```

```
cd GitDemo
```

```
git status
```

```
git branch GitWork
```

```
git checkout GitWork
```

```
echo <message>Hello from GitWork</message> > hello.xml
```

```
type hello.xml
```

```
git status
```

```
git add hello.xml
```

```
git commit -m "Added hello.xml in GitWork"
```

```
git checkout main
```

```
echo <message>Hello from Main Branch</message> > hello.xml
```

```
type hello.xml
```

```
git add hello.xml
```

```
git commit -m "Added hello.xml in main"
```

```
git log --oneline --graph --decorate --all
```

```
git diff main GitWork
```

```
git merge GitWork
```

```
type hello.xml
```

```
git add hello.xml
```

```
git commit -m "Resolved merge conflict"
```

```
git status
```

```
echo *.bak >> .gitignore
```

```
type .gitignore
```

```
git add .gitignore
```

```
git commit -m "Updated gitignore for backup files"
```

```
git branch
```

```
git branch -d GitWork
```

```
git log --oneline --graph --decorate
```

git push origin main

What Was Implemented

- Opened the existing Git repository.
- Verified that the repository was in a clean state.
- Created a new branch named **GitWork**.
- Created a file named **hello.xml** in the branch.
- Added and committed the file in the branch.
- Switched back to the **main** branch.
- Created the same file with different content.
- Committed the changes in the **main** branch.
- Compared the differences between the branch and the main branch.
- Merged the branch into the main branch.
- Observed the merge conflict markers generated by Git.
- Resolved the conflict manually by editing hello.xml.
- Added and committed the resolved file.
- Updated .gitignore to ignore backup files (*.bak).
- Committed the updated .gitignore.
- Listed all available branches.
- Deleted the merged branch.
- Viewed the commit history using the Git graph.
- Pushed the updated repository to GitHub.

Result

The Merge Conflict Resolution Hands-On Lab was successfully completed. A merge conflict was intentionally created by modifying the same file in different branches. The conflict was identified, resolved manually, committed successfully, and the repository was updated. The merged branch was deleted, .gitignore was updated to ignore backup files, and the final changes were pushed to GitHub, demonstrating the complete Git merge conflict resolution workflow.

```
Command Prompt
* dac47c5 (HEAD -> main) Added hello.xml in main branch
* ab149e5 (GitWork) Added hello.xml in GitWork branch
* 465d088 (origin/main) Added branch file in GitNewBranch
* 59102ff Added gitignore file
* f5cd28d Added welcome.txt

C:\Users\vanka\Desktop\GitDemo>git diff main GitWork
diff --git a/hello.xml b/hello.xml
index 295da9e..2959b21 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1,1 @@
- <message>Hello from Main branch</message>
+ <message>Hello from GitWork branch</message>

C:\Users\vanka\Desktop\GitDemo>git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.

C:\Users\vanka\Desktop\GitDemo>type hello.xml
<<<<< HEAD
<message>Hello from Main branch</message>
=====
<message>Hello from GitWork branch</message>
>>>>> GitWork

C:\Users\vanka\Desktop\GitDemo>echo ^<message^>Hello from Main branch^</message^> > hello.xml

C:\Users\vanka\Desktop\GitDemo>echo ^<message^>Hello from GitWork branch^</message^> >> hello.xml

C:\Users\vanka\Desktop\GitDemo>type hello.xml
<message>Hello from Main branch</message>
<message>Hello from GitWork branch</message>

C:\Users\vanka\Desktop\GitDemo>git add hello.xml

C:\Users\vanka\Desktop\GitDemo>git commit -m "Resolved merge conflict in hello.xml"
[main ff0e33c] Resolved merge conflict in hello.xml
```

```
Command Prompt

MOVING

e ^E j ^N CR * Forward one line (or N lines).
y ^Y k ^K ^P * Backward one line (or N lines).
ESC-j * Forward one file line (or N file lines).
ESC-k * Backward one file line (or N file lines).
f ^F ^V SPACE * Forward one window (or N lines).
b ^B ESC-v * Backward one window (or N lines).
z * Forward one window (and set window to N).
w * Backward one window (and set window to N).
ESC-SPACE * Forward one window, but don't stop at end-of-file.
ESC-b * Backward one window, but don't stop at beginning-of-file.
d ^D * Forward one half-window (and set half-window to N).
u ^U * Backward one half-window (and set half-window to N).
ESC-~ * Right one half screen width (or N positions).
ESC-~ * Left one half screen width (or N positions).
ESC-} ^RightArrow Right to last column displayed.
ESC-{ ^LeftArrow Left to first column.
F Forward forever; like "tail -f".
ESC-F Like F but stop when search pattern is found.
r ^R ^L Repaint screen.
R Repaint screen, discarding buffered input.

-----
Default "window" is the screen height.
Default "half-window" is half of the screen height.
-----

SEARCHING

/pattern * Search forward for (N-th) matching line.
?pattern * Search backward for (N-th) matching line.
n * Repeat previous search (for N-th occurrence).
N * Repeat previous search in reverse direction.
ESC-n * Repeat previous search, spanning files.
ESC-N * Repeat previous search, reverse dir. & spanning files.
^O^N ^On * Search forward for (N-th) OSC8 hyperlink.
^O^P ^Op * Search backward for (N-th) OSC8 hyperlink.
^O^L ^Ol * Jump to the currently selected OSC8 hyperlink.
ESC-u Undo (toggle) search highlighting.
ESC-U Clear search highlighting.
&pattern * Display only matching lines.
```

```
Command Prompt x + ~
```

```
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
  
GitWork  
* main  
~  
  
C:\Users\vanka\Desktop\GitDemo>git log --oneline --graph --decorate  
* 3717ee1 (HEAD -> main) Updated gitignore for backup files  
 * f08e33c Resolved merge conflict in hello.xml  
/  
 * ab149e5 (GitWork) Added hello.xml in GitWork branch  
 | dac47c5 Added hello.xml in main branch  
/  
 * 465d088 (origin/main) Added branch file in GitNewBranch  
 * 59102ff Added gitignore file  
 * f5cd28d Added welcome.txt  
~  
  
C:\Users\vanka\Desktop\GitDemo>git push origin main  
Enumerating objects: 14, done.  
Counting objects: 100% (14/14), done.  
Delta compression using up to 12 threads  
Compressing objects: 100% (9/9), done.  
Writing objects: 100% (12/12), 1.27 KiB | 433.00 KiB/s, done.  
Total 12 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (4/4), completed with 1 local object.  
To https://github.com/Adharshinivankayala/GitDemo.git  
   465d088..3717ee1  main -> main  
  
C:\Users\vanka\Desktop\GitDemo>
```